Kadane's Array

* Sliding → only (+)ve ; Target 'K' given → Subarray with target 'K'
* Prefix → Both (+) & (−)ve & '0' ; Target 'K' given → Subarray with target 'K'
* Kadane's → (+)ve, (−)ve & '0' ; No target given → Subarray with max Sum/Product

Kadane's Skeleton template →

```
Sum = max(n[i], Sum + n[i])
ans = max(ans, Sum)
```

Leet-53 Max. Subarray

```
int Sum = nums[0]
int maxSum = nums[0]
int j=0
for (i=1 to n)
    Sum += num[i]
    while (Sum<0)
        Sum -= num[j]
        j++
    maxSum = max(Sum, maxSum)
return maxSum;
```

Template for Sliding win

Sliding ✗ Failed

Why → Sliding window won't work

Ex → [-10, -5]

Ans → -5

But according to our code

Sum always less than 0

So, j++ & fully exhaust

Correct approach

Kadane's ✓

```
int Sum = n[0]
int maxSum = nums[0]
for (int i=0; i<n; i++){
    Sum += nums[i]
    maxSum = max(maxSum, Sum);
    if (Sum<0)
        Sum = 0
}
return maxSum
```

[-10, -5]

Sum = 0
maxS = -10

i=0

Sum = 0 + (-10) = -10
maxSum = max(-10, -10) = -10
if (Sum < 0) Sum = 0

i=1

Sum = 0 − 5 = −5
maxSum = max(-10, -5) = -5
if (Sum < 0) Sum = 0

return maxSum = -5 ✓

T: O(n)
S: O(1)

(*) one more shorter version check leet code

Ajanta PRODUCTS Officer

Leet-152 Max product Subarray

Brute → T: $O(n^2)$ S: $O(1)$ → check leetcode

Optimal 1 → T: $O(n)$ S: $O(1)$

next optimal ? only

```
int maxP = nums[0]
int minP = nums[0]
int ans = nums[0]

for (i=1 to n)
    if(n[i]<0)
        swap(maxP, minP)
    maxP = max(n[i], maxP * n[i])
    minP = min(n[i], minP * n[i])
    ans = max(maxP, ans);
return ans;
```

Always mind to swap at beginning

if -ve no arrive swap max & min P before multiplying

then multiply & choose max-min

Optimal 2 → T: $O(n)$ S: $O(1)$

2nd

whenever '0' comes reset pref = 1, suff = 1

[2, 3, 0, 4, 5]

Pref → 2×1=2, 2×3=6, 6×0=0 → Pref=1, 1×4=4, 4×5=20 → max

Suff → 5×1=5, 5×4=20 → max, 20×0=0 → Suff=1, 1×3=3, 3×2=6

```
int maxP = nums[0]
int Prefix = 1      → Take product from left
int Suffix = 1      → Take Product from right

for(i=0 to n)
    if (Prefix ==0) Prefix = 1;
    if (Suffix ==0) Suffix = 1;

    Prefix *= nums[i];
    Suffix *= nums[n-1-i];

return maxP = max(maxP, max(Prefix, Suffix))
```

why to use this edge case

1st check this

[2, 3, -2, 4]

Prefix (left) = 2×1=2, 2×3=6, 6×-2=-12, -12×4=-48 → store max from left

Suffix (right) = 4×1=4, 4×-2=-8, -8×3=-24, -24×2=-48 → store max from right

max(Suff, Pref) → 6 Ans

like this we can find Prefix & Suff in single loop

Leet-1749 Max Abs Sum of Arry

maxS = 0
minS = 0
ans = 0

> Sum is required so take '0' instead of num[0]

for (i=0 to n)

- maxS = max(n[i], maxS + n[i]) → largest +ve sum
- minS = min(n[i], minS + n[i]) → largest -ve sum
- ans = max(ans, max(abs(maxS), abs(minS)))

Basic template of Kadane's.

return ans;

T: O(n)
S: O(1)

Leet-918 Max Sum Circular SubArray

int maxS = 0
int minS = 0
} finds local maxim & local minim sum upto n[i]

int maxSum = n[0]
int minSum = n[0]
} find the maximest & minimest sum throughout the entire Arry

int tSum = 0
→ Sum of all elements of array

for (i=0 to n)
maxS = max(n[i], maxS + n[i])
minS = min(n[i], minS + n[i])

maxSum = max(maxSum, maxS)
minSum = min(minSum, minS)

t += n[i]

if (maxSum < 0) → All elements negative
return maxSum;

return max(maxSum, tsum - minSum) → (i)

(iii) -3, -2, -1
maxSum = -1
tSum = -6
minSum = -6
* tSum - minSum = 0
* max(maxSum, tsum - minSum)
= max(-1, 0) = 0
* But ans will be -1 ⇒ maxSum

(ii) 2, -10, 4, -40, 3, 4
minimest sum
tsum
remove minSum from tsum
will get wrapped sum of
[2, 3, 4]

(ii) 2, -10, 14, -40, 3, 4
minimest
wrapped = tsum - minSum
= 2 + (-10) + 14 + 3 + 4